
Representações Distribuídas de Sentenças e Documentos

Quoc Le
Tomas Mikolov

QVL@GOOGLE.COM
TMIKOLOV@GOOGLE.COM

Google Inc, 1600 Amphitheatre Parkway, Mountain View, CA 94043

Resumo

Muitos algoritmos de aprendizado de máquina exigem entrada a ser representada como uma característica de comprimento fixo vetor. Quando se trata de textos, um dos mais Uma característica comum de comprimento fixo é o modelo de saco de palavras. Apesar de sua popularidade, os recursos de saco de palavras Apresentam duas grandes desvantagens: perdem a ordem das palavras e ignoram a semântica. das palavras. Por exemplo, "poderoso", "forte" e "Paris" estão igualmente distantes. Neste artigo, nós Propomos o Paragraph Vector, um algoritmo não supervisionado que aprende representações de características de comprimento fixo a partir de trechos de texto de comprimento variável, como: frases, parágrafos e documentos. Nosso algoritmo representa cada documento por um vetor denso, treinado para prever palavras no documento. Sua construção confere ao nosso algoritmo a capacidade de prever e analisar dados. Potencial para superar as fragilidades dos modelos de saco de palavras. Resultados empíricos mostram que os Vetores de Parágrafos superam os modelos de saco de palavras. bem como outras técnicas para representação de texto. Finalmente, alcançamos novos resultados de última geração em diversas classificações de texto e análise de sentimentos. tarefas de análise.

1. Introdução

A classificação e o agrupamento de textos desempenham um papel importante. Em muitas aplicações, por exemplo, recuperação de documentos, pesquisa na web, filtragem de spam. No cerne dessas aplicações estão algoritmos de aprendizado de máquina, como regressão logística ou K-means. Esses algoritmos normalmente exigem que a entrada de texto seja... ser representado como um vetor de comprimento fixo. Talvez o mais A representação vetorial comum de comprimento fixo para textos é o saco de palavras ou saco de n-gramas (Harris, 1954) devido ao seu Simplicidade, eficiência e, muitas vezes, uma precisão surpreendente.

No entanto, o modelo de saco de palavras (BOW) apresenta muitas desvantagens.

Anais da 31ª Conferência Internacional sobre Máquinas Aprendizagem, Pequim, China, 2014. JMLR: W&CP volume 32. Direitos autorais 2014 do(s) autor(es).

tags. A ordem das palavras se perde, resultando em frases diferentes. podem ter exatamente a mesma representação, desde que o As mesmas palavras são usadas. Embora o método "bag-of-n-grams" considere a ordem das palavras em um contexto curto, ele sofre com a falta de dados. esparsidade e alta dimensionalidade. Os modelos de saco de palavras e saco de n-gramas têm pouca noção da semântica do palavras ou, mais formalmente, as distâncias entre as palavras. Isso significa que as palavras "poderoso", "forte" e "Paris" são igualmente distantes, apesar de, semanticamente, "poderoso" dever estar mais próximo de "forte" do que de "Paris".

Neste artigo, propomos o Paragraph Vector, uma estrutura não supervisionada que aprende vetores distribuídos contínuos.

representações para trechos de texto. Os textos podem ser de de comprimento variável, desde frases até documentos. O nome "Paragraph Vector" serve para enfatizar o fato de que... O método pode ser aplicado a textos de comprimento variável. Pode ser desde uma frase ou sentença até um documento extenso.

Em nosso modelo, a representação vetorial é treinada para ser útil na previsão de palavras em um parágrafo. Mais precisamente, nós Concatenar o vetor do parágrafo com vários vetores de palavras de um parágrafo e prever a palavra seguinte no parágrafo. dado o contexto. Tanto os vetores de palavras quanto os vetores de parágrafos são treinados pelo método de descida de gradiente estocástico e retropropagação (Rumelhart et al., 1986). Enquanto os vetores de parágrafo são O que é único entre os parágrafos é que os vetores de palavras são compartilhados. Em No tempo de previsão, os vetores de parágrafo são inferidos fixando os vetores de palavras e treinando o novo vetor de parágrafo. até a convergência.

Nossa técnica é inspirada pelo trabalho recente em aprendizado de representações vetoriais de palavras usando redes neurais (Bengio et al., 2006; Collobert & Weston, 2008; Mnih & Hinton, 2008; Turian et al., 2010; Mikolov e outros, 2013a;c). Em sua formulação, cada palavra é representada por um vetor que é concatenado ou calculado a média com outra palavra vetores em um contexto, e o vetor resultante é usado para prever outras palavras no contexto. Por exemplo, a rede neural modelo de linguagem de rede proposto em (Bengio et al., 2006) utiliza a concatenação de vários vetores de palavras anteriores para forma a entrada de uma rede neural e tenta prever a próxima palavra. O resultado é que, após o treinamento do modelo, Os vetores de palavras são mapeados em um espaço vetorial de forma que

Palavras semanticamente semelhantes têm representações vetoriais semelhantes (por exemplo, "forte" é próximo de "poderoso").

Seguindo essas técnicas bem-sucedidas, pesquisadores tentaram expandir os modelos para ir além do nível da palavra, buscando representações em nível de frase ou sentença (Mitchell & Lapata, 2010; Zanzotto et al., 2010; Yessenalina & Cardie, 2011; Grefenstette et al., 2013; Mikolov et al., 2013c). Por exemplo, uma abordagem simples consiste em usar uma média ponderada de todas as palavras do documento.

Uma abordagem mais sofisticada consiste em combinar os vetores de palavras em uma ordem dada por uma árvore de análise sintática de uma sentença, usando operações de matriz-vetor (Socher et al., 2011b). Ambas as abordagens têm fragilidades. A primeira abordagem, a média ponderada dos vetores de palavras, perde a ordem das palavras da mesma forma que os modelos padrão de saco de palavras. A segunda abordagem, que usa uma árvore de análise sintática para combinar vetores de palavras, demonstrou funcionar apenas para sentenças, pois depende da análise sintática.

O Paragraph Vector é capaz de construir representações de sequências de entrada de comprimento variável. Ao contrário de algumas abordagens anteriores, ele é geral e aplicável a textos de qualquer tamanho: frases, parágrafos e documentos. Não requer ajustes específicos da função de ponderação de palavras nem depende de árvores de análise sintática. Mais adiante neste artigo, apresentaremos experimentos em diversos conjuntos de dados de referência que demonstram as vantagens do Paragraph Vector. Por exemplo, na tarefa de análise de sentimentos, alcançamos resultados de última geração, superiores a métodos complexos, com uma melhoria relativa de mais de 16% na taxa de erro. Em uma tarefa de classificação de texto, nosso método supera de forma convincente os modelos de saco de palavras, apresentando uma melhoria relativa de cerca de 30%.

2. Algoritmos

Começamos por discutir métodos anteriores para aprender vetores de palavras. Esses métodos são a inspiração para nossos métodos de vetores de parágrafo.

2.1. Aprendendo a Representação Vetorial de Palavras

Esta seção apresenta o conceito de representação vetorial distribuída de palavras. Uma estrutura bem conhecida para aprendizado de vetores de palavras é mostrada na Figura 1. A tarefa consiste em prever uma palavra a partir das outras palavras em um contexto.

Nesse contexto, cada palavra é mapeada para um vetor único, representado por uma coluna em uma matriz W . A coluna é indexada pela posição da palavra no vocabulário. A concatenação ou soma dos vetores é então usada como característica para a previsão da próxima palavra em uma frase.

De forma mais formal, dada uma sequência de palavras de treinamento, $w_1, w_2, w_3, \dots, w_T$, o objetivo do modelo de vetor de palavras é

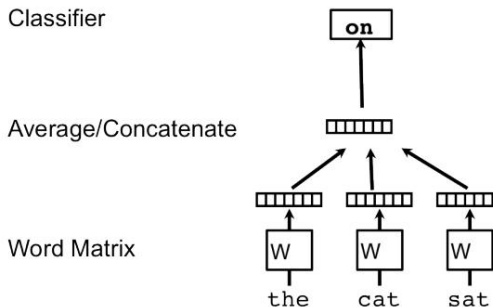


Figura 1. Uma estrutura para aprendizado de vetores de palavras. O contexto de três palavras ("o", "gato" e "sentado") é usado para prever a quarta palavra ("em"). As palavras de entrada são mapeadas para colunas da matriz W para prever a palavra de saída.

é maximizar a probabilidade logarítmica média

$$\frac{1}{T} \sum_{t=k}^T \log p(w_t | w_{1:k}, \dots, w_{t+k})$$

A tarefa de previsão é normalmente realizada por meio de um classificador multiclasse, como o softmax. Nesse caso, temos

$$p(w_t | w_{1:k}, \dots, w_{t+k}) = \frac{e^{y_{wt}}}{\sum_{i=1}^V e^{y_{wi}}}$$

Cada y_i é a probabilidade logarítmica não normalizada para cada palavra de saída i , calculada como

$$y = b + U h(w_{1:k}, \dots, w_{t+k}; W) \quad (1)$$

onde U e b são os parâmetros softmax. h é construído por meio da concatenação ou média de vetores de palavras extraídos de W .

Na prática, a função softmax hierárquica (Morin & Bengio, 2005; Mnih & Hinton, 2008; Mikolov et al., 2013c) é preferida à softmax para treinamento rápido. Em nosso trabalho, a estrutura da softmax hierárquica é uma árvore de Huffman binária, onde códigos curtos são atribuídos a palavras frequentes. Este é um bom truque para acelerar o treinamento, pois as palavras comuns são acessadas rapidamente.

Este uso do código binário de Huffman para a hierarquia é o mesmo que em (Mikolov et al., 2013c).

Os vetores de palavras baseados em redes neurais são geralmente treinados usando o método do gradiente descendente estocástico, onde o gradiente é obtido por meio de retropropagação (Rumelhart et al., 1986). Esse tipo de modelo é comumente conhecido como modelo neural de linguagem (Bengio et al., 2006). Uma implementação específica de um algoritmo baseado em redes neurais para o treinamento de vetores de palavras está disponível em code.google.com/p/word2vec/ (Mikolov et al., 2013a).

Após a convergência do treinamento, palavras com significados semelhantes são mapeadas para posições similares no espaço vetorial.

Representações Distribuídas de Sentenças e Documentos

Por exemplo, “poderoso” e “forte” estão próximos um do outro, enquanto “poderoso” e “Paris” estão mais distantes. A diferença entre vetores de palavras também carrega significado. Por exemplo, os vetores de palavras podem ser usados para responder a perguntas de analogia usando álgebra vetorial simples: “Rei” - “homem” + “mulher” = “Rainha” (Mikolov et al., 2013d). Também é possível aprender uma matriz linear para traduzir palavras e frases entre idiomas (Mikolov et al., 2013b).

Essas propriedades tornam os vetores de palavras atraentes para muitas tarefas de processamento de linguagem natural, como modelagem de linguagem (Bengio et al., 2006; Mikolov, 2012), compreensão de linguagem natural (Collobert & Weston, 2008; Zhila et al., 2013), tradução automática estatística (Mikolov et al., 2013b; Zou et al., 2013), compreensão de imagens (Frome et al., 2013) e extração relacional (Socher et al., 2013a).

2.2. Vetor de Parágrafo: Um modelo de memória distribuída

Nossa abordagem para aprender vetores de parágrafo é inspirada nos métodos de aprendizado de vetores de palavras. A inspiração reside no fato de que os vetores de palavras são solicitados a contribuir para uma tarefa de predição da próxima palavra na frase. Assim, apesar de os vetores de palavras serem inicializados aleatoriamente, eles podem eventualmente capturar a semântica como um resultado indireto da tarefa de predição. Usaremos essa ideia em nossos vetores de parágrafo de maneira similar. Os vetores de parágrafo também são solicitados a contribuir para a tarefa de predição da próxima palavra, dados diversos contextos amostrados do parágrafo.

Em nossa estrutura de Vetores de Parágrafo (ver Figura 2), cada parágrafo é mapeado para um vetor único, representado por uma coluna na matriz D, e cada palavra também é mapeada para um vetor único, representado por uma coluna na matriz W. Os vetores de parágrafo e de palavra são calculados em média ou concatenados para prever a próxima palavra em um contexto. Nos experimentos, usamos a concatenação como método para combinar os vetores.

Mais formalmente, a única mudança neste modelo em comparação com a estrutura do vetor de palavras está na equação 1, onde h é construído a partir de W e D .

O marcador de parágrafo pode ser considerado como outra palavra. Ele funciona como uma memória que guarda o que está faltando no contexto atual — ou no tópico do parágrafo. Por esse motivo, costumamos chamar esse modelo de Modelo de Memória Distribuída de Vetores de Parágrafo (PV-DM).

Os contextos têm comprimento fixo e são amostrados a partir de uma janela deslizante sobre o parágrafo. O vetor do parágrafo é compartilhado entre todos os contextos gerados a partir do mesmo parágrafo, mas não entre parágrafos diferentes. A matriz de vetores de palavras W , no entanto, é compartilhada entre parágrafos. Ou seja, o vetor para “powerful” é o mesmo para todos os parágrafos.

Os vetores de parágrafo e os vetores de palavra são treinados usando

O método de descida de gradiente estocástico utiliza o método de retropropagação. A cada passo dessa descida, podemos amostrar um contexto de comprimento fixo de um parágrafo aleatório, calcular o gradiente de erro a partir da rede mostrada na Figura 2 e usar esse gradiente para atualizar os parâmetros do nosso modelo.

No momento da previsão, é necessário realizar uma etapa de inferência para calcular o vetor do parágrafo para um novo parágrafo. Isso também é obtido por meio do método do gradiente descendente. Nessa etapa, os parâmetros para o restante do modelo, os vetores de palavras W e os pesos softmax, são fixados.

Suponha que existam N parágrafos no corpus, M palavras no vocabulário, e que desejemos aprender vetores de parágrafo de forma que cada parágrafo seja mapeado para p dimensões e cada palavra seja mapeada para q dimensões. Nesse caso, o modelo terá um total de $N \times p + M \times q$ parâmetros (excluindo os parâmetros softmax). Embora o número de parâmetros possa ser grande quando N é grande, as atualizações durante o treinamento são tipicamente esparsas e, portanto, eficientes.

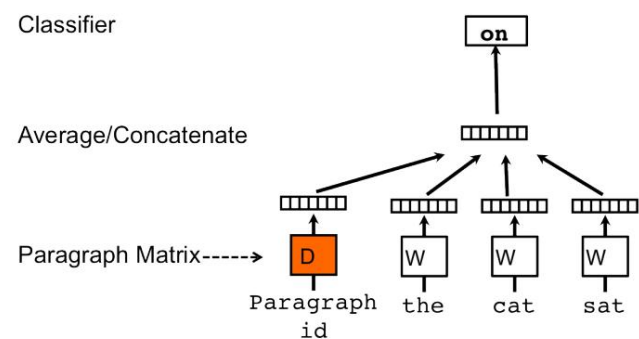


Figura 2. Uma estrutura para aprendizado de vetores de parágrafo. Essa estrutura é semelhante à apresentada na Figura 1; a única diferença é o token de parágrafo adicional que é mapeado para um vetor por meio da matriz D . Nesse modelo, a concatenação ou média desse vetor com um contexto de três palavras é usada para prever a quarta palavra. O vetor de parágrafo representa a informação ausente no contexto atual e pode funcionar como uma memória do tópico do parágrafo.

Após o treinamento, os vetores de parágrafo podem ser usados como características do parágrafo (por exemplo, em vez de ou em adição ao modelo de saco de palavras). Podemos alimentar essas características diretamente em técnicas convencionais de aprendizado de máquina, como regressão logística, máquinas de vetores de suporte ou K-means.

Em resumo, o algoritmo em si possui duas etapas principais: 1) treinamento para obter vetores de palavras W , pesos softmax U e b e vetores de parágrafo D em parágrafos já vistos; e 2) a “etapa de inferência” para obter vetores de parágrafo D para novos parágrafos (nunca vistos antes) adicionando mais colunas em D e aplicando o método do gradiente descendente em D , mantendo W , U e b fixos. Usamos D para fazer uma previsão sobre alguns rótulos específicos usando um classificador padrão, por exemplo, regressão logística.

Vantagens dos vetores de parágrafo: Uma importante vantagem dos vetores de parágrafo é que eles são aprendidos a partir de dados não rotulados e, portanto, podem funcionar bem para tarefas que não possuem dados rotulados suficientes.

Os vetores de parágrafo também abordam algumas das principais fragilidades dos modelos de saco de palavras. Primeiro, eles herdam uma propriedade importante dos vetores de palavras: a semântica das palavras. Nesse espaço, “poderoso” está mais próximo de “forte” do que de “Paralelo”. A segunda vantagem dos vetores de parágrafo é que eles levam em consideração a ordem das palavras, pelo menos em um contexto pequeno, da mesma forma que um modelo de n-gramas com um n grande faria. Isso é importante, porque o modelo de n-gramas preserva muitas informações do parágrafo, incluindo a ordem das palavras. Dito isso, nosso modelo talvez seja melhor do que um modelo de saco de n-gramas, porque este último criaria uma representação de altíssima dimensionalidade que tende a generalizar mal.

2.3. Vetor de parágrafo sem ordenação de palavras: Saco de palavras distribuído

O método acima considera a concatenação do vetor do parágrafo com os vetores das palavras para prever a próxima palavra em uma janela de texto. Outra maneira é ignorar as palavras de contexto na entrada, mas forçar o modelo a prever palavras amostradas aleatoriamente do parágrafo na saída. Na prática, isso significa que, a cada iteração do gradiente descendente estocástico, amostramos uma janela de texto, depois amostramos uma palavra aleatória dessa janela e formulamos uma tarefa de classificação dado o vetor do parágrafo. Essa técnica é ilustrada na

Figura 3. Denominamos essa versão de Vetor de Parágrafo com Saco de Palavras Distribuído (PV-DBOW), em oposição à versão com Memória Distribuída (PV-DM) da seção anterior.

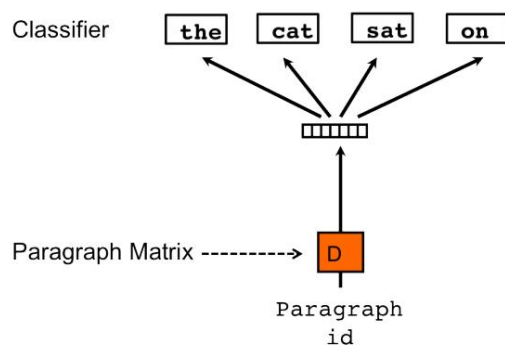


Figura 3. Versão Distributed Bag of Words dos vetores de parágrafo. Nesta versão, o vetor do parágrafo é treinado para prever as palavras em uma pequena janela.

Além de ser conceitualmente simples, este modelo requer menos armazenamento de dados. Precisamos armazenar apenas os pesos softmax, em vez dos pesos softmax e vetores de palavras do modelo anterior. Este modelo também é semelhante ao

Modelo Skip-gram em vetores de palavras (Mikolov et al., 2013c).

Em nossos experimentos, cada vetor de parágrafo é uma combinação de dois vetores: um aprendido pelo vetor de parágrafo padrão com memória distribuída (PV-DM) e outro aprendido pelo vetor de parágrafo com saco de palavras distribuído (PV-DBOW). O PV-DM sozinho geralmente funciona bem para a maioria das tarefas (com desempenhos de última geração), mas sua combinação com o PV-DBOW geralmente é mais consistente em diversas tarefas que testamos e, portanto, é fortemente recomendada.

3. Experimentos

Realizamos experimentos para melhor compreender o comportamento dos vetores de parágrafo. Para isso, avaliamos o desempenho do Paragraph Vector em dois problemas de compreensão de texto que exigem representações vetoriais de parágrafos de comprimento fixo: análise de sentimentos e recuperação de informação.

Para a análise de sentimentos, utilizamos dois conjuntos de dados: o conjunto de dados Stanford Sentiment Treebank (Socher et al., 2013b) e o conjunto de dados IMDB (Maas et al., 2011). Os documentos nesses conjuntos de dados diferem significativamente em extensão: cada exemplo no conjunto de dados de Socher et al. (Socher et al., 2013b) é uma única frase, enquanto cada exemplo no conjunto de dados de Maas et al. (Maas et al., 2011) consiste em várias frases.

Também testamos nosso método em uma tarefa de recuperação de informações, cujo objetivo é decidir se um documento deve ser recuperado a partir de uma consulta.

3.1. Análise de Sentimentos com o Conjunto de Dados Stanford Sentiment Treebank

Conjunto de dados: Este conjunto de dados foi proposto inicialmente por (Pang & Lee, 2005) e posteriormente ampliado por (Socher et al., 2013b) como referência para análise de sentimentos. Ele contém 11.855 frases extraídas do site de críticas de filmes Rotten Tomatoes.

O conjunto de dados consiste em três conjuntos: 8544 frases para treinamento, 2210 frases para teste e 1101 frases para validação (ou desenvolvimento).

Cada frase no conjunto de dados possui um rótulo que varia de muito negativo a muito positivo em uma escala de 0,0 a 1,0.

Os rótulos são gerados por anotadores humanos usando o Amazon Mechanical Turk.

O conjunto de dados inclui rótulos detalhados para frases e subfrases na mesma escala. Para isso, Socher et al. (Socher et al., 2013b) utilizaram o Stanford Parser (Klein & Manning, 2003) para analisar cada frase e decompor as subfrases. As subfrases foram então rotuladas por anotadores humanos da mesma forma que as frases. No total, o conjunto de dados contém 239.232 frases rotuladas. O conjunto de dados pode ser baixado em:

<http://nlp.Stanford.edu/sentiment/>

Representações Distribuídas de Sentenças e Documentos

Tarefas e Linhas de Base: Em (Socher et al., 2013b), os autores propõem duas maneiras de realizar benchmarking. Primeiro, pode-se...

Considere uma tarefa de classificação refinada de 5 vias, onde

Os rótulos são {Muito Negativo, Negativo, Neutro, Positivo, Muito Positivo} ou uma tarefa de classificação binária de granularidade grosseira, onde os rótulos são {Negativo, Positivo}.

Outro eixo de variação diz respeito a se devemos rotular a frase inteira ou todas as expressões da frase. Neste caso,

Em nosso trabalho, consideramos apenas a rotulagem de frases completas.

Socher et al. (Socher et al., 2013b) aplicam vários métodos para este conjunto de dados e descobrir que seu Tensor Neural Recursivo

A rede funciona muito melhor do que o modelo de saco de palavras.

Pode-se argumentar que isso ocorre porque as críticas de cinema são frequentemente

A concisão e a composicionalidade desempenham um papel importante na decisão sobre se a avaliação é positiva ou negativa, assim como...

semelhança entre palavras, dado o tamanho bastante diminuto de

o conjunto de treinamento.

Protocolos experimentais: Seguimos o protocolo experimental.

protocolos conforme descrito em (Socher et al., 2013b). Para fazer

Utilizando os dados rotulados disponíveis, em nosso modelo, cada subfrase é tratada como uma sentença independente e aprendemos.

As representações para todas as subfrases no conjunto de treinamento.

Após aprendermos as representações vetoriais para as frases de treinamento e suas subfrases, as alimentamos em uma regressão logística para aprender um preditor da classificação do filme.

No momento do teste, congelamos a representação vetorial para cada um.

palavra, e aprenda as representações para as frases usando

descida de gradiente. Uma vez que as representações vetoriais para o

Após o aprendizado das frases de teste, utilizamos a regressão logística para prever a classificação do filme.

Em nossos experimentos, realizamos a validação cruzada do tamanho da janela usando o conjunto de validação, e o tamanho ideal da janela é 8.

O vetor apresentado ao classificador é uma concatenação de dois vetores, um de PV-DBOW e outro de PV-DM.

Em PV-DBOW, as representações vetoriais aprendidas têm 400

dimensões. Em PV-DM, as representações vetoriais aprendidas

Possuem 400 dimensões tanto para palavras quanto para parágrafos.

Para prever a oitava palavra, concatenamos os vetores do parágrafo e os vetores das sete palavras. Caracteres especiais como „!?” são

tratado como uma palavra normal. Se o parágrafo tiver menos de 9

Nas palavras, adicionamos um símbolo especial de palavra NULL antes de cada uma.

Resultados: Apresentamos as taxas de erro de diferentes métodos em

Tabela 1. O primeiro destaque desta tabela é que os modelos de saco de palavras ou saco de n-gramas (NB, SVM, BiNB) têm um desempenho ruim. Simplesmente calcular a média dos vetores de palavras (como em um modelo de saco de palavras) não melhora os resultados. Isso é

porque os modelos de saco de palavras não consideram como cada

A frase é composta (por exemplo, a ordem das palavras) e, portanto,

não conseguem reconhecer muitos fenômenos linguísticos sofisticados, como o sarcasmo. Os resultados também mostram que

Tabela 1. Desempenho do nosso método comparado a outras abordagens no conjunto de dados Stanford Sentiment Treebank. O erro

As taxas de outros métodos são relatadas em (Socher et al., 2013b).

Modelo	Taxa de erro (Positivo/ (Grão fino-negativo))	Taxa de erro
Naive Bayes (Socher et al., 2013b)	18,2%	59,0%
SVMs (Socher et al., 2013b)	20,6%	59,3%
Bigram Naïve Bayes (Socher et al., 2013b)	16,9%	58,1%
Média vetorial de palavras (Socher et al., 2013b)	19,9%	67,3%
Rede Neural Recursiva (Socher et al., 2013b)	17,6%	56,8%
Vetor matricial-RNN (Socher et al., 2013b)	17,1%	55,6%
Rede Tensorial Neural Recursiva (Socher et al., 2013b)	14,6%	54,3%
Vetor de parágrafo	12,2%	51,3%

métodos mais avançados (como a Rede Neural Recursiva (Socher et al., 2013b)), que requerem análise sintática e

Levando em consideração a composicionalidade, o desempenho é muito melhor.

Nosso método apresenta melhor desempenho do que todas essas linhas de base, por exemplo,

redes recursivas, apesar de não exigirem nova análise sintática. Na tarefa de classificação de granularidade grossa, nosso

O método apresenta uma melhoria absoluta de 2,4% em termos de

taxas de erro. Isso se traduz em uma melhoria relativa de 16%.

3.2. Além de uma frase: Análise de sentimento com Conjunto de dados IMDB

Algumas das técnicas anteriores funcionam apenas em frases,

mas não parágrafos/documentos com várias frases. Para

instância, Rede Tensorial Neural Recursiva (Socher et al.,

2013b) baseia-se na análise sintática de cada frase e

Não está claro como combinar as representações em vários níveis.

frases. Portanto, essas técnicas são restritas ao trabalho

em frases, mas não em parágrafos ou documentos.

Nosso método não requer análise sintática, portanto pode produzir

uma representação para um documento longo composto por muitos

frases. Essa vantagem torna nosso método mais geral

do que algumas outras abordagens. O experimento a seguir, realizado com o conjunto de dados IMDB, demonstra essa vantagem.

Conjunto de dados: O conjunto de dados IMDB foi proposto inicialmente por Maas.

et al. (Maas et al., 2011) como referência para análise de sentimentos. O conjunto

de dados consiste em 100.000 críticas de filmes coletadas.

do IMDB. Um aspecto fundamental deste conjunto de dados é que cada

A crítica do filme tem várias frases.

As 100.000 críticas de filmes estão divididas em três conjuntos de dados:

25.000 instâncias de treinamento rotuladas, 25.000 instâncias de teste rotuladas e 50.000 instâncias de treinamento não rotuladas. Existem dois tipos de rótulos: Positivo e Negativo. Esses rótulos são balanceado tanto no conjunto de treinamento quanto no conjunto de teste. O conjunto de dados Pode ser baixado em <http://ai.Stanford.edu/amaas/data/sentiment/index.html>

Protocolos experimentais: Aprendemos os vetores de palavras e vetores de parágrafo usando 75.000 documentos de treinamento (25.000 (50.000 instâncias rotuladas e 50.000 instâncias não rotuladas). O parágrafo Em seguida, são fornecidos vetores para as 25.000 instâncias rotuladas. por meio de uma rede neural com uma camada oculta com 50 unidades e um classificador logístico para aprender a prever o sentimento.¹

No momento do teste, dada uma frase de teste, congelamos novamente o restante da rede e aprender os vetores de parágrafo para o teste revisões por descida de gradiente. Uma vez que os vetores são aprendidos, Nós as alimentamos através da rede neural para prever o sentimento das avaliações.

Os hiperparâmetros do nosso modelo de vetor de parágrafo são selecionados da mesma forma que na tarefa anterior. Em particular, validamos o tamanho da janela, sendo o tamanho ideal de 10 palavras. O vetor apresentado ao

O classificador é uma concatenação de dois vetores, um proveniente de PV-DBOW e outro de PV-DM. Em PV-DBOW, o classificador aprendido As representações vetoriais têm 400 dimensões. Em PV-DM, As representações vetoriais aprendidas têm 400 dimensões para tanto palavras quanto documentos. Para prever a décima palavra, nós Concatena os vetores de parágrafo e os vetores de palavra. Caracteres especiais como „!?” são tratados como palavras normais. Se o documento tiver menos de 9 palavras, adicionamos um espaço em branco antes dele. Símbolo especial de palavra NULL.

Resultados: Os resultados do Paragraph Vector e de outras linhas de base são apresentados na Tabela 2. Como pode ser observado na Tabela: para documentos longos, os modelos de saco de palavras têm bom desempenho. bastante bem e é difícil melhorá-los usando vetores de palavras. A melhoria mais significativa ocorreu Em 2012, no trabalho de (Dahl et al., 2012), eles combinaram um modelo de Máquinas de Boltzmann Restritas com o modelo de saco de palavras. A combinação dos dois modelos resultou em uma melhoria de aproximadamente 1,5% em termos de taxas de erro.

Outra melhoria significativa advém do trabalho de (Wang & Manning, 2012). Entre muitas variações, eles Testado, o NBSVM em recursos de bigramas funciona melhor e produz os seguintes resultados. uma melhoria considerável de 2% em termos de erro avaliar.

O método descrito neste artigo é a única abordagem. Isso vai muito além da barreira de uma taxa de erro de 10%.

¹Em nossos experimentos, a rede neural apresentou um desempenho melhor. do que um classificador logístico linear nesta tarefa.

Atinge 7,42%, o que representa uma melhoria absoluta de 1,3% (ou uma melhoria relativa de 15%) em relação ao melhor resultado anterior. resultado de (Wang & Manning, 2012).

Tabela 2. Desempenho do Paragraph Vector em comparação com outros abordagens no conjunto de dados IMDB. As taxas de erro de outros métodos são relatados em (Wang & Manning, 2012).

Modelo	Taxa de erro
BoW (bnc) (Maas et al., 2011)	12,20%
BoW (bȳt'c) (Maas et al., 2011)	11,77%
LDA (Maas et al., 2011)	32,58%
Full+BoW (Maas et al., 2011)	11,67%
Completo+Não rotulado+BoW (Maas et al., 2011)	11,11%
WRRBM (Dahl et al., 2012)	12,58%
WRRBM + BoW (bnc) (Dahl et al., 2012)	10,77%
MNB-uni (Wang & Manning, 2012)	16,45%
MNB-bi (Wang & Manning, 2012)	13,41%
SVM-uni (Wang & Manning, 2012)	13,05%
SVM-bi (Wang & Manning, 2012)	10,84%
NBSVM-uni (Wang & Manning, 2012)	11,71%
NBSVM-bi (Wang & Manning, 2012)	8,78%
Vetor de parágrafo	7,42%

3.3. Recuperação de Informação com Vetores de Parágrafo

Voltamos nossa atenção para uma tarefa de recuperação de informações que Requer representações de parágrafos de comprimento fixo.

Aqui, temos um conjunto de dados de parágrafos nos 10 primeiros resultados. retornado por um mecanismo de busca dado cada um dos 1.000.000 mais consultas populares. Cada um desses parágrafos também é conhecido como Um "trecho" que resume o conteúdo de uma página da web. e como uma página da web corresponde à consulta.

A partir dessa coleção, derivamos um novo conjunto de dados para testar o vetor. representações de parágrafos. Para cada consulta, criamos um conjunto de três parágrafos: os dois parágrafos são resultados de a mesma consulta, enquanto o terceiro parágrafo é aleatório parágrafo selecionado do restante da coleção (retornado) como resultado de uma consulta diferente). Nosso objetivo é identificar Qual dos três parágrafos são resultados da mesma consulta? Para alcançar esse objetivo, utilizaremos vetores de parágrafo e calcularemos as distâncias dos parágrafos. Uma representação melhor é uma que alcança uma pequena distância para pares de parágrafos do mesma consulta e uma grande distância para pares de parágrafos de consultas diferentes.

Aqui está um exemplo de três parágrafos, onde o primeiro parágrafo deve estar mais próximo do segundo parágrafo do que o terceiro. terceiro parágrafo:

- **Parágrafo 1:** chamadas de (000) 000 - 0000 . 3913
Foram relatadas chamadas originadas deste número. De acordo com 4 relatos, a identidade do chamador é da American Airlines.

• **Parágrafo 2:** você quer descobrir quem te ligou?
de +1 000 - 000 - 0000 +1 0000000000 ou (000) 000 - 0000 ? veja relatórios e compartilhe informações você
tenho informações sobre essa pessoa que ligo

• **Parágrafo 3:** pacientes da clínica Allina Health para o seu
Para sua comodidade, você pode pagar sua consulta na clínica Allina Health.
Pague sua conta online. Pague sua conta da clínica agora, tire suas dúvidas e...
respostas...

Os trios são divididos em três conjuntos: 80% para treinamento, 10% para validação e 10% para teste. Qualquer método que exija aprendizado será treinado no conjunto de treinamento, enquanto seu Os hiperparâmetros serão selecionados no conjunto de validação.

Avaliamos quatro métodos para calcular características de parágrafos: saco de palavras, saco de bigramas e média de palavras. vetores e vetor de parágrafo. Para melhorar o modelo de saco de bigramas, Também aprendemos uma matriz de ponderação de forma que a distância entre os dois primeiros parágrafos seja minimizada, enquanto que A distância entre o primeiro e o terceiro parágrafo é maximizada (o fator de ponderação entre as duas perdas é um hiperparâmetro).

Registramos o número de vezes que cada método produz resultados. menor distância para os dois primeiros parágrafos do que para o primeiro e o terceiro parágrafo. Um erro é cometido se um método não produz a métrica de distância desejável em um trio de parágrafos.

Os resultados do Paragraph Vector e de outras linhas de base são apresentados na Tabela 3. Nesta tarefa, constatamos que a ponderação TF-IDF apresenta melhor desempenho do que as contagens brutas e, portanto, utilizamos apenas... Apresentar os resultados dos métodos com ponderação TF-IDF.

Os resultados mostram que o Paragraph Vector funciona bem e Proporciona uma melhoria relativa de 32% em termos de taxa de erro. O fato de o método de vetor de parágrafo superar significativamente o método de saco de palavras e bigramas sugere que o método proposto é útil para capturar a semântica do parágrafo. Digite o texto.

Tabela 3. Desempenho do Paragraph Vector e do modelo de saco de palavras. modelos na tarefa de recuperação de informação. "Saco de bigramas ponderado" é o método em que aprendemos uma matriz linear W em características de bigramas TF-IDF que maximiza a distância entre os primeiros e o terceiro parágrafo e minimiza a distância entre o primeiro e segundo parágrafos.

Modelo	Taxa de erro
Média vetorial	10,25%
Saco de palavras	8,10%
Saco de bigrams	7,28%
Saco ponderado de bigramas	5,67%
Vetor de parágrafo	3,82%

3.4. Algumas observações adicionais

Realizamos experimentos adicionais para compreender vários aspectos dos modelos. Aqui estão algumas observações.

- O PV-DM é consistentemente melhor que o PV-DBOW. O PV-DM sozinho pode alcançar resultados próximos a muitos outros. neste artigo (ver Tabela 2). Por exemplo, no IMDB, O PV-DM atinge apenas 7,63%. A combinação de PV-DM e PV-DBOW costumam funcionar de forma consistentemente melhor (7,42% no IMDB) e, portanto, são recomendados.
- O uso de concatenação em PV-DM costuma ser melhor do que soma. No IMDB, PV-DM com soma só pode alcançar 8,06%. Talvez isso ocorra porque o modelo perde o Informações para encomenda.
- É melhor validar o tamanho da janela de forma cruzada. Uma boa Em muitas aplicações, estima-se que o tamanho da janela esteja entre 5 e 12. No IMDB, variando os tamanhos das janelas entre Os valores 5 e 12 fazem com que a taxa de erro flutue em 0,7%.
- A criação de vetores para parágrafos pode ser cara, mas é possível. em paralelo durante o teste. Em média, nossa implementação leva 30 minutos para calcular os vetores de parágrafo do conjunto de teste do IMDB, usando uma máquina com 16 núcleos. (25.000 documentos, cada documento em média tem (230 palavras).

4. Trabalhos relacionados

Representações distribuídas para palavras foram propostas inicialmente. em (Rumelhart et al., 1986) e se tornaram um sucesso paradigma, especialmente para modelagem estatística de linguagem (Elman, 1990; Bengio et al., 2006; Mikolov, 2012). Vetores de palavras têm sido usados em aplicações de PNL, como representação de palavras, reconhecimento de entidades nomeadas, desambiguação de sentidos de palavras, análise sintática, etiquetagem e tradução automática (Collobert & Weston, 2008; Turney & Pantel, 2010; Turian). e outros, 2010; Collobert et al., 2011; Socher et al., 2011b; Huang et al., 2012; Zou et al., 2013).

Representar frases é uma tendência recente e tem recebido muita atenção. atenção (Mitchell & Lapata, 2010; Zanzotto et al., 2010; Yessenalina e Cardie, 2011; Grefenstette et al., 2013; Mikolov e outros, 2013c). Nessa direção, estilo autoencoder Os modelos também foram usados para modelar parágrafos (Maas) e outros, 2011; Larochelle e Lauly, 2012; Srivastava e outros, 2013).

Representações distribuídas de frases e sentenças são também o foco de Socher et al. (Socher et al., 2011a;c; 2013b). Seus métodos normalmente exigem análise sintática e são demonstrou funcionar para representações em nível de sentença. E isso Não é óbvio como estender seus métodos além de um único exemplo. frases. Seus métodos também são supervisionados e, portanto, exigem mais dados rotulados para funcionar bem. Vetor de parágrafo,

Representações Distribuídas de Sentenças e Documentos

Em contraste, é principalmente não supervisionado e, portanto, pode funcionar bem com menos dados rotulados.

Nossa abordagem de calcular os vetores do parágrafo por meio do método do gradiente descendente apresenta semelhanças com um paradigma bem-sucedido em visão computacional (Perronnin & Dance, 2007; Perronnin et al., 2010) conhecidos como núcleos de Fisher (Jaakkola & Hausler, 1999). A construção básica dos núcleos de Fisher é a Vetor gradiente sobre um modelo generativo não supervisionado.

5. Discussão

Descrevemos o Paragraph Vector, um método de aprendizado não supervisionado. Algoritmo que aprende representações vetoriais para trechos de texto de comprimento variável, como frases e documentos. As representações vetoriais são aprendidas para prever as palavras circundantes em contextos amostrados do parágrafo.

Nossos experimentos em diversas tarefas de classificação de texto, como Conjuntos de dados de análise de sentimentos Stanford Treebank e IMDB demonstrar que o método é competitivo com o estado da arte métodos. O bom desempenho demonstra os méritos de vetores de parágrafo na captura da semântica de parágrafos. De fato, os vetores de parágrafo têm o potencial de superar muitas das fragilidades dos modelos de saco de palavras.

Embora o foco deste trabalho seja representar textos, nosso O método pode ser aplicado para aprender representações de dados sequenciais. Em domínios não textuais onde a análise sintática não está disponível, esperamos que o Vetor de Parágrafo seja uma alternativa viável para modelos de saco de palavras e saco de n-gramas.

Referências

Bengio, Yoshua, Schwenk, Holger, Senecal, Jean-Sebastien, Morin, Frederic e Gauvain, Jean-Luc. Modelos de linguagem probabilísticos neurais. Em Inovações em Aprendizado de Máquina, pp. 137–186. Springer, 2006.

Collobert, Ronan e Weston, Jason. Uma arquitetura unificada. Para processamento de linguagem natural: Redes neurais profundas com aprendizado multitarefa. Em Anais da 25ª Conferência Internacional de Aprendizado de Máquina, pp. 160–167. ACM, 2008.

Collobert, Ronan, Weston, Jason, Bottou, Leon, Karlen, Michael, Kavukcuoglu, Koray e Kuksa, Pavel. Processamento de linguagem natural (quase) do zero. O Revista de Pesquisa em Aprendizado de Máquina, 12:2493–2537, 2011.

Dahl, George E., Adams, Ryan P. e Larochelle, Hugo. Treinamento de Máquinas de Boltzmann Restritas em observações de palavras. In Conferência Internacional sobre Aprendizado de Máquina, 2012.

Elman, Jeff. Encontrando estrutura no tempo. Em Ciência Cognitiva, pp. 179–211, 1990.

Frome, Andrea, Corrado, Greg S., Shlens, Jonathon, Ben-gio, Samy, Dean, Jeffrey, Ranzato, Marc'Aurelio e Mikolov, Tomas. DeViSE: Um modelo de incorporação visual-semântica profunda. Em Advances in Neural Information Processing Systems, 2013.

Grefenstette, E., Dinu, G., Zhang, Y., Sadrzadeh, M., e Baroni, M. Aprendizado de regressão em múltiplas etapas para semântica distribucional composicional. In Conferência sobre Métodos Empíricos em Processamento de Linguagem Natural, 2013.

Harris, Zellig. Estrutura distribucional. Word, 1954.

Huang, Eric, Socher, Richard, Manning, Christopher e Ng, Andrew Y. Melhorando as representações de palavras por meio de Contexto global e múltiplos protótipos de palavras. In Anais da 50ª Reunião Anual da Associação Para Linguística Computacional: Artigos Longos - Volume 1, pp. 873–882. Associação para Linguística Computacional, 2012.

Jaakkola, Tommi e Hausler, David. Explorando modelos generativos em classificadores discriminativos. Em Advances em Sistemas de Processamento de Informação Neural 11, pp. 487–493, 1999.

Klein, Dan e Manning, Chris D. Análise sintática precisa sem lexicalização. In Anais da Associação de Linguística Computacional, 2003.

Larochelle, Hugo e Lauly, Stanislas. Um modelo de tópico autorregressivo neural. Em Advances in Neural Information. Sistemas de Processamento, 2012.

Maas, Andrew L., Daly, Raymond E., Pham, Peter T., Huang, Dan, Ng, Andrew Y. e Potts, Christopher. Aprendizagem de vetores de palavras para análise de sentimentos. In Anais da 49ª Reunião Anual da Associação para Linguística Computacional, 2011.

Mikolov, Tomas. Modelos de linguagem estatísticos baseados em Redes Neurais. Tese de doutorado, Universidade Tecnológica de Brno, 2012.

Mikolov, Tomas, Chen, Kai, Corrado, Greg e Dean, Jeffrey. Estimativa eficiente de representações de palavras no espaço vetorial. arXiv preprint arXiv:1301.3781, 2013a.

Mikolov, Tomas, Le, Quoc V., e Sutskever, Ilya. Explorando semelhanças entre línguas para tradução automática. CoRR, abs/1309.4168, 2013b.

Mikolov, Tomas, Sutskever, Ilya, Chen, Kai, Corrado, Greg, e Dean, Jeffrey. Representações distribuídas de frases e sua composicionalidade. Em Advances on Neural Information Processing Systems, 2013c.

Representações Distribuídas de Sentenças e Documentos

- Mikolov, Tomas, Yih, Scott Wen-tau e Zweig, Geoffrey.
Regularidades linguísticas nas representações de palavras no espaço contínuo. Em NAACL HLT, 2013d.
- Mitchell, Jeff e Lapata, Mirella. Composição em modelos distribucionais de semântica. Ciência Cognitiva, 2010.
- Mnih, Andriy e Hinton, Geoffrey E. Um modelo de linguagem distribuído hierárquico escalável. Em Advances in Sistemas de Processamento de Informação Neural, pp. 1081–1088, 2008.
- Morin, Frederic e Bengio, Yoshua. Modelo de linguagem de rede neural probabilística hierárquica. Em Anais do Workshop Internacional sobre Inteligência Artificial e Estatísticas, pp. 246–252, 2005.
- Pang, Bo e Lee, Lillian. Vendo estrelas: Explorando a classe relações para categorização de sentimentos em relação a escalas de classificação. In Anais da Associação de Linguística Computacional, pp. 115–124, 2005.
- Perronnin, Florent e Dance, Christopher. Grãos de Fisher sobre vocabulários visuais para categorização de imagens. Em IEEE Conferência sobre Visão Computacional e Reconhecimento de Padrões, 2007.
- Perronnin, Florent, Liu, Yan, Sanchez, Jorge e Poirier, Hervé. Recuperação de imagens em larga escala com compressão. Vetores de Fisher. Em Conferência IEEE sobre Visão Computacional e Reconhecimento de Padrões, 2010.
- Rumelhart, David E, Hinton, Geoffrey E, e Williams, Ronald J. Aprendendo representações por retropropagação erros. Nature, 323(6088):533–536, 1986.
- Socher, Richard, Huang, Eric H., Pennington, Jeffrey, Manning, Chris D., e Ng, Andrew Y. Agrupamento dinâmico e desdobramento de autoencoders recursivos para paráfrase detecção. Em Advances in Neural Information Processing Systems, 2011a.
- Socher, Richard, Lin, Cliff C, Ng, Andrew e Manning, Chris. Analisando cenas naturais e linguagem natural com redes neurais recursivas. Em Anais do 28º Conferência Internacional sobre Aprendizado de Máquina (ICML-11), pp. 129–136, 2011b.
- Socher, Richard, Pennington, Jeffrey, Huang, Eric H, Ng, Andrew Y., e Manning, Christopher D. Autoencoders recursivos semi-supervisionados para prever distribuições de sentimentos. In Anais da Conferência sobre Métodos Empíricos no Processamento de Linguagem Natural, 2011c.
- Socher, Richard, Chen, Danqi, Manning, Christopher D., e Ng, Andrew Y. Raciocínio com redes tensoriais neurais para preenchimento de bases de conhecimento. Em Advances in Sistemas de Processamento de Informação Neural, 2013a.
- Socher, Richard, Perelygin, Alex, Wu, Jean Y., Chuang, Jason, Manning, Christopher D., Ng, Andrew Y. e Potts, Christopher. Modelos profundos recursivos para composicionalidade semântica em um treebank de sentimentos. Em Conferência sobre Métodos Empíricos no Processamento de Linguagem Natural, 2013b.
- Srivastava, Nitish, Salakhutdinov, Ruslan e Hinton, Geoffrey. Modelagem de documentos com máquinas Boltzmann profundas. Em Incerteza em Inteligência Artificial, 2013.
- Turian, Joseph, Ratinov, Lev e Bengio, Yoshua. Palavra Representações: um método simples e geral para aprendizado semi-supervisionado. Em Anais da 48ª Conferência Anual Encontro da Associação de Linguística Computacional, pp. 384–394. Associação de Linguística Computacional, 2010.
- Turney, Peter D. e Pantel, Patrick. Da frequência para Significado: Modelos de espaço vetorial da semântica. Revista de Pesquisa em Inteligência Artificial, 2010.
- Wang, Sida e Manning, Chris D. Linhas de base e bigramas: Classificação simples e eficaz de sentimentos e textos. Em Anais da 50ª Reunião Anual da Associação para Linguística Computacional, 2012.
- Yessenalina, Ainur e Cardie, Claire. Composição Modelos de espaço matricial para análise de sentimentos. In Conferência sobre Métodos Empíricos em Processamento de Linguagem Natural, 2011.
- Zanzotto, Fabio, Korkontzelos, Ioannis, Fallucchi, Francesca e Manandhar, Suresh. Estimativa linear Modelos para semântica distribucional composicional. Em COLING, 2010.
- Zhila, A., Yih, WT, Meek, C., Zweig, G. e Mikolov, T. Combinando modelos heterogêneos para medir a similaridade relacional. Em NAACL HLT, 2013.
- Zou, Will, Socher, Richard, Cer, Daniel e Manning, Christopher. Incorporação de palavras bilíngues para tradução automática baseada em frases. Em Conferência sobre Empirismo Métodos em Processamento de Linguagem Natural, 2013.